Optimization of Prompt Design: A Hybrid Approach Combining Regeneration and Selection

WANG YU 王語

京都大学情報学研究科 下平・本多研究室

2025/02

Contents

Introduction

What is Prompt Engineering?

- Guides a large language model to **perform specific** tasks by **designing input** (prompt).
- To improve the **quality** and **accuracy** of output through specific input forms.
- An efficient way to fine-tune models **without** the need for **fine-tuning**.
- Widely used in NLP tasks (such as question answering, summary generation, logical reasoning, etc.).

Contents

Related Work

APE[2] :

- Selects **high-performing** prompts from a large pool.
- **Disadvantages:**
  1. Strong dependence on the evaluation mechanism
  2. The quality of generated prompts is unstable
  3. Lack of iterative optimization

PromptBreeder[3]:

- A self-referential evolutionary framework that continuously **mutates** and **refines** prompts.
- **Disadvantages:**
  1. High computational cost
  2. Overfitting risk
  3. Generation efficiency issues

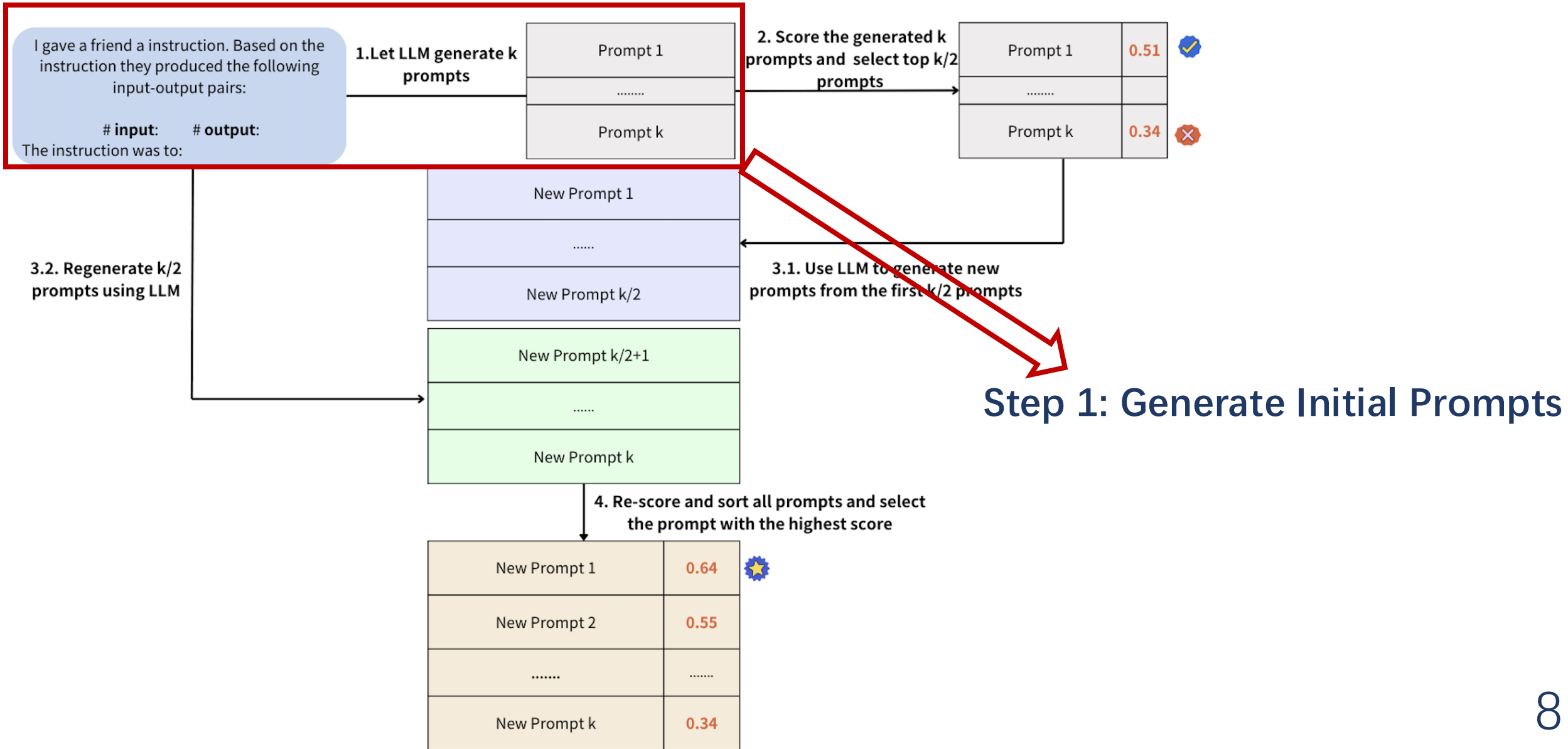
Contents

Method- Motivation

- Existing prompt optimization methods often focus too much on either improving **quality** or exploring **diverse** prompts, but **not both**.
  - APE : do not refine or improve prompts further, limiting adaptability.
  - PromptBreeder: requires a lot of computational power & lead to unnecessary complexity.

- What we want:
  - Both **refine** good prompts and **explore** new ones efficiently,
  - Striking a **balance** between **performance** and **adaptability**
  - Without excessive computation.

Method- Overview

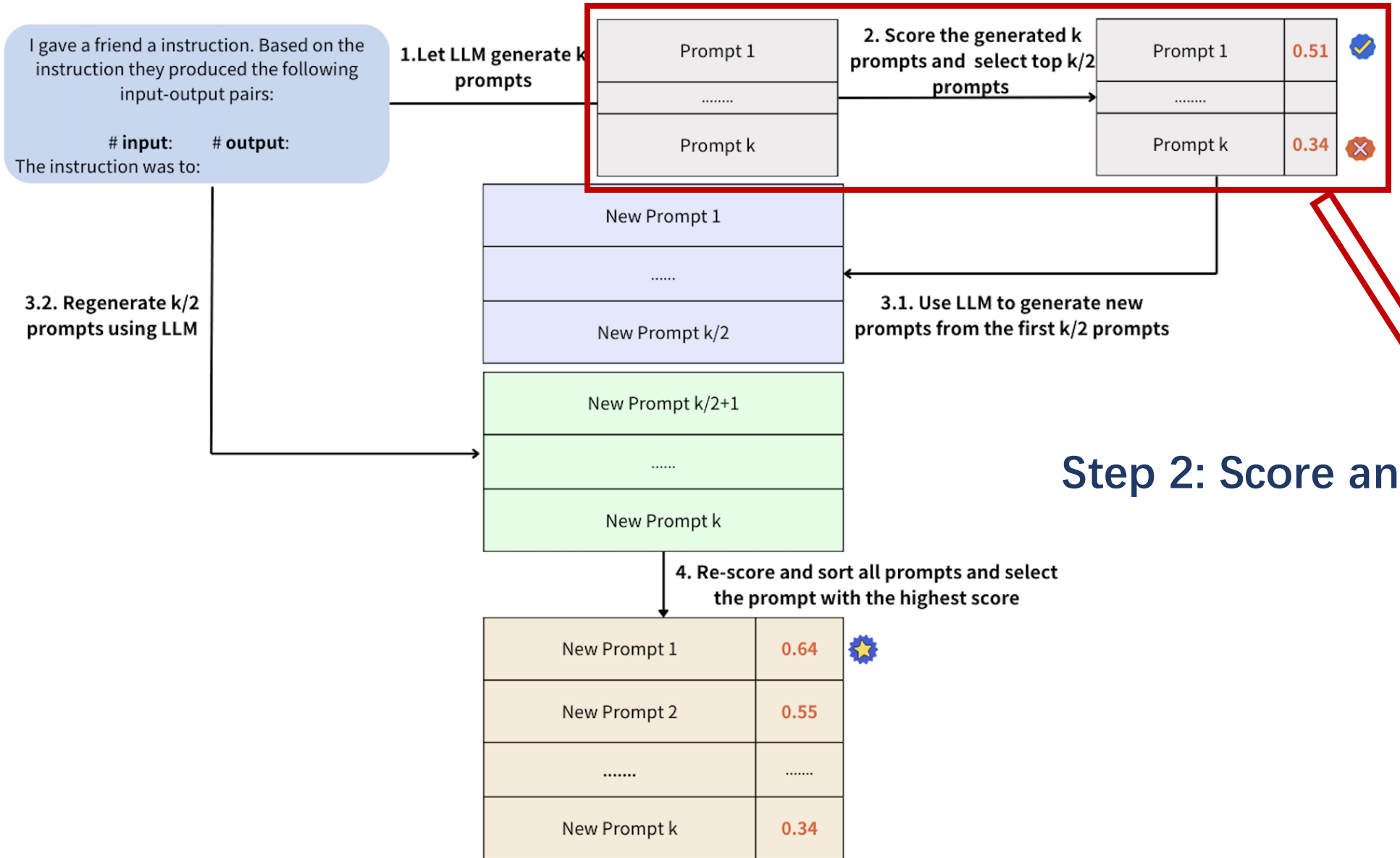


Step 1: Generate Initial Prompts

Method- Step 1

- **How do we start?**
  - Generate **k diverse prompts** using predefined templates.
  - Ensures broad exploration of possible task formulations.

Method- Overview



Step 2: Score and Select k/2 Prompts

Method- Step 2

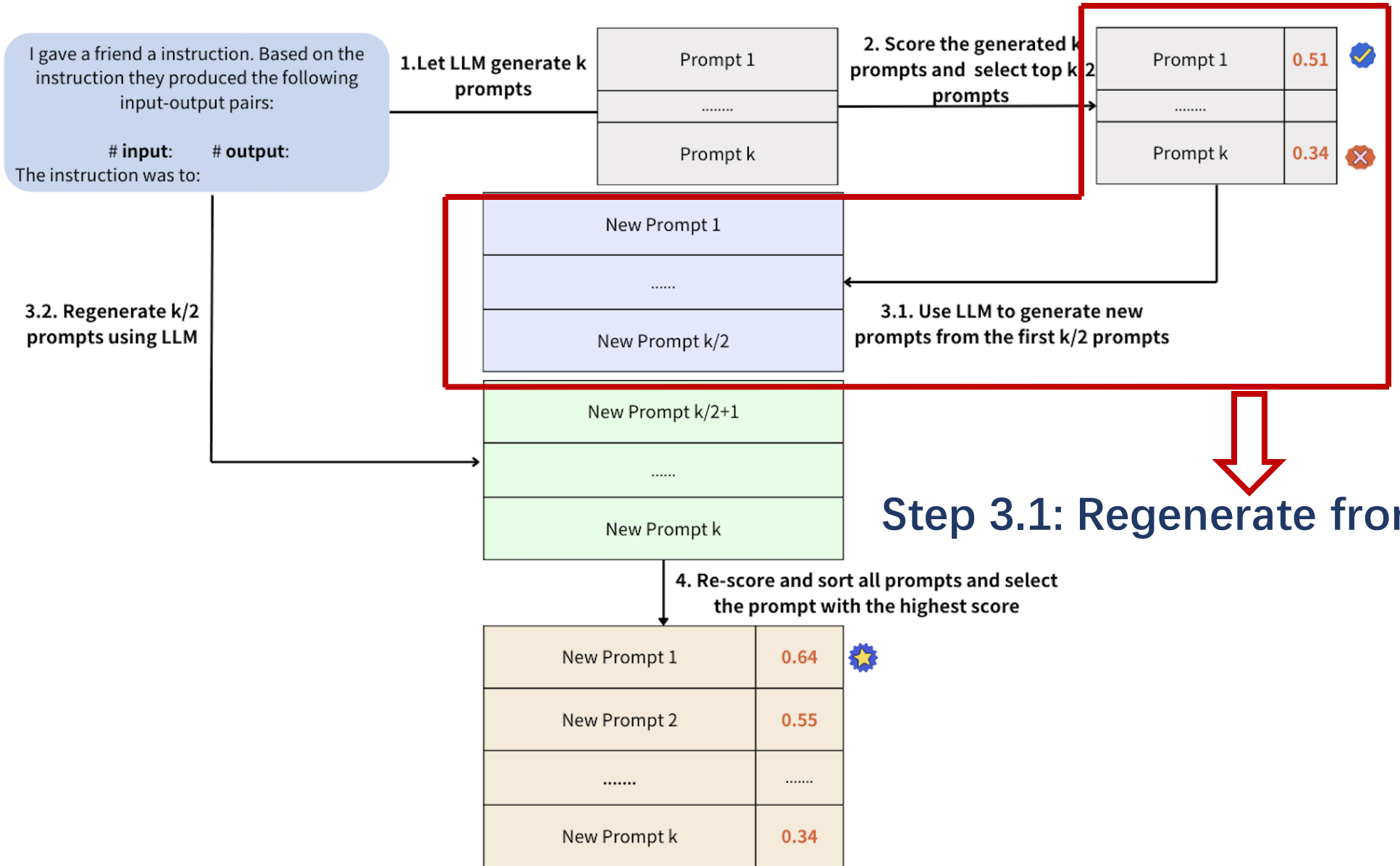
- **How do we determine the best prompts?**
  - Score each prompt using **LLM-driven evaluation**.
  - Retain the **top k/2 prompts** with the highest scores.

K prompts generated in Step 1

LLM

Scored prompts
Choose the top k/2

Method- Overview

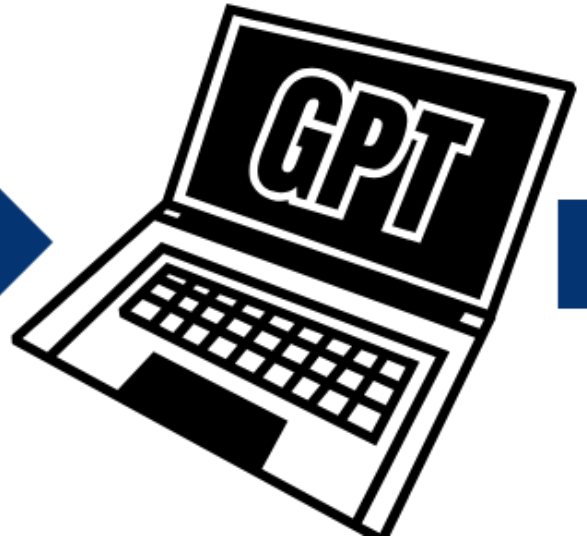


Step 3.1: Regenerate from Top k/2 Prompts

Method- Step 3.1

- **How do we improve high-quality prompts?**
  - Regenerate **k/2 new prompts** from the top-performing ones.
  - **Regeneration Method:**
    - Direct Mutation[3] : Small refinements
    - Hypermutation[3] : Large variations for diversity.
    - APE-inspired Regeneration[2] : Combining refinement with task alignment.

1. create the opposite or antonym of the given word.
2. find the opposite or contrasting word for each given input.



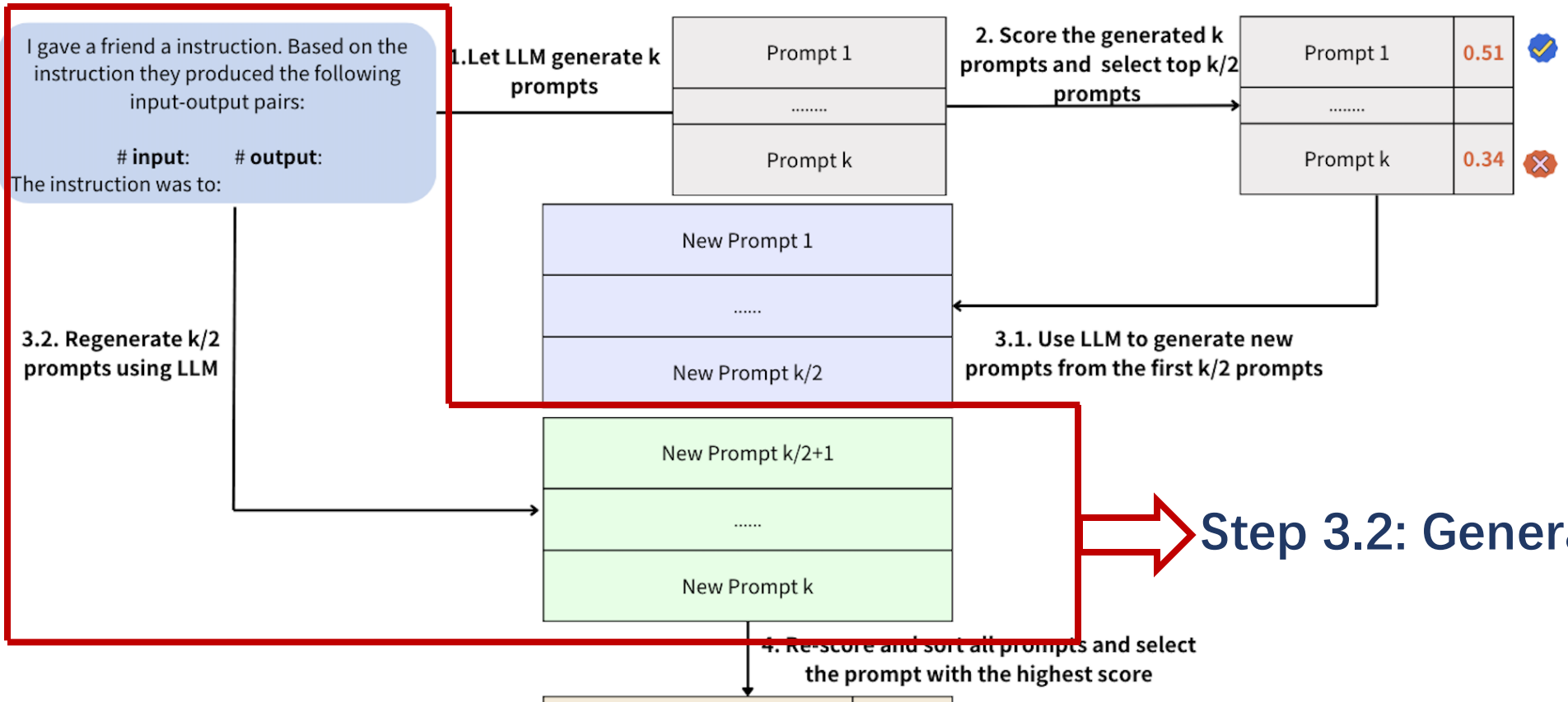
1. find the opposite or antonym of each given word.
2. provide the opposite or antonym of each given word.

The top k/2 prompts selected in Step 2.

LLM

Regenerate k/2 prompts

Method- Overview



Step 3.2: Generate k/2 New Prompts

Method- Step 3.2

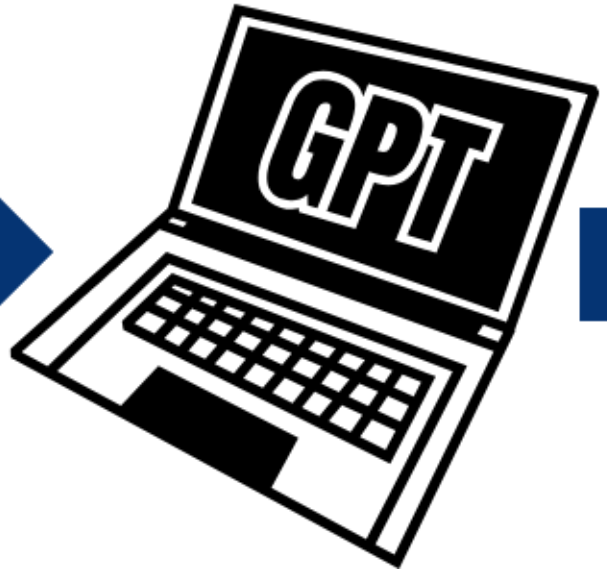
- **How do we ensure diversity?**
  - Generate another **k/2 prompts** from scratch using the **initial prompt generation method**.
  - Ensures exploration of different structures and perspectives.

Template:

I gave a friend a instruction. Based on the instruction they produced the following input output pairs:

Input:{INPUT} **Output**:{OUTPUT}

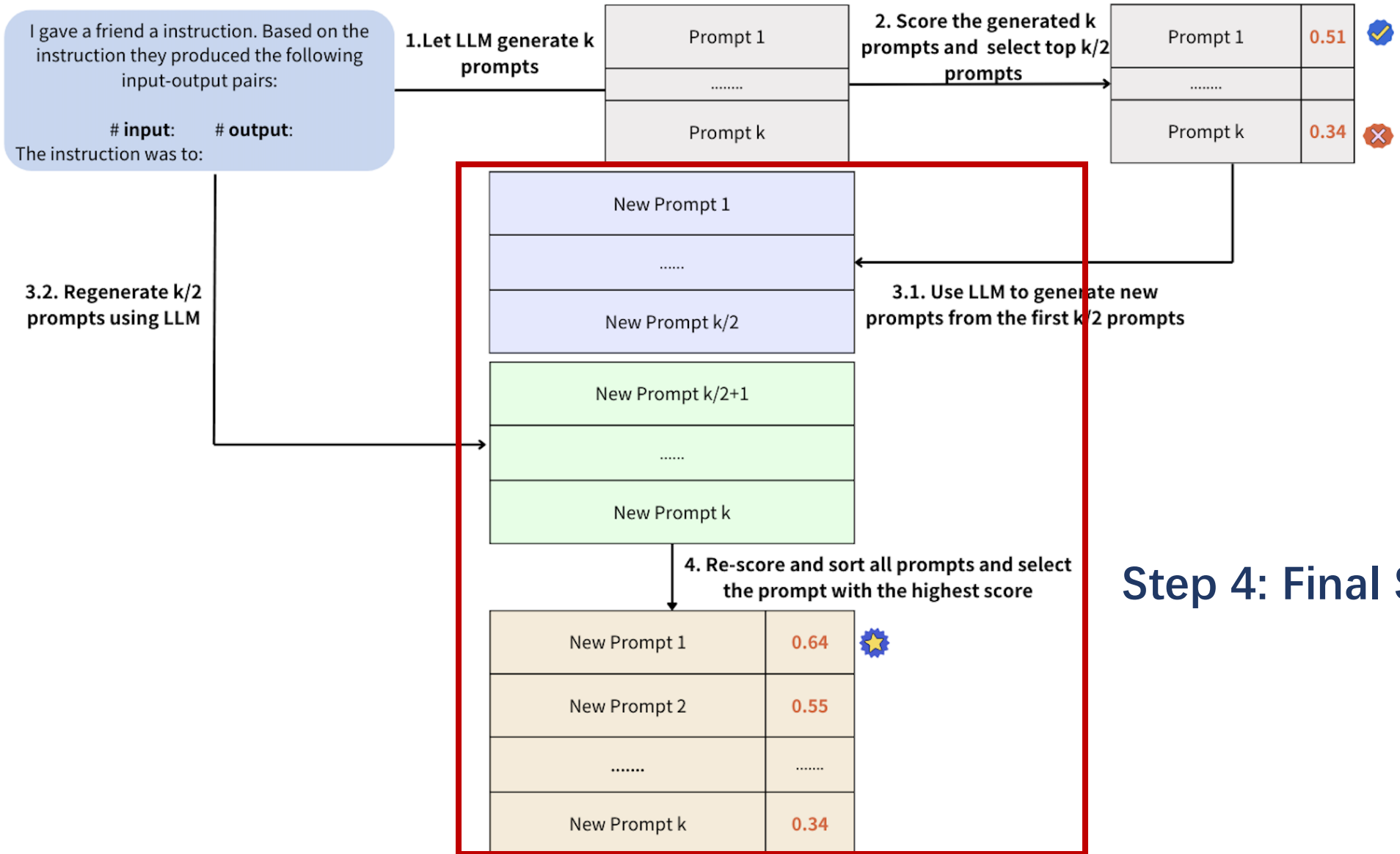
The instruction was to:



1. **Determine the opposite of every word.**
2. **Locate the antonym or reverse of the provided term.**

Same template as Step 1 **LLM** **Newly generated k/2 prompts**

Method- Overview



Step 4: Final Scoring and Selection

Method- Step 4

- **How do we choose the best prompt?**
  - Score the final **k prompts** again.
  - Select the **highest-scoring** prompt as the final output.

New k/2 prompts regenerated in Step 3.1

1. find the opposite or antonym of each given word.
2. provide the opposite or antonym of each given word.

Input:{INPUT} Output:{OUTPUT}

1. Determine the opposite of every word.
2. Locate the antonym or reverse of the provided term.

New k/2 prompts generated in Step 3.2

LLM

1. Determine the opposite of every word.
2. find the opposite or antonym of each given word.
3. provide the opposite or antonym of each given word.
4. Locate the antonym or reverse of the provided term.

0.95

0.8

0.75

0.65

Score all newly generated prompts
Choose the highest-scoring prompt

Method

- **Contributions and Advantages:**
  - **Balance between exploration and optimization:**
    - Optimization of high-scoring prompts **ensures quality**.
    - Generation of new prompts **maintains innovation**.
  - **Avoid overfitting:**
    - Through multiple rounds of screening and generation, **reduce** the **risk** of prompts being too dependent on specific patterns.
  - **Improve resource efficiency:**
    - **Reduce** the **waste** of resources in invalid generation and evaluation.

Contents

Experiment - Setup

- Datasets are formatted as **input and output pairs**.
- Each dataset is divided into two parts:
  - **Score** the generated prompts during the evaluation process
  - **Test** the **accuracy** of the selected prompts
- A **subset** of input-output pairs is **randomly** selected from the scoring part of the dataset to serve as input for **generating prompts**.
- Use **GPT-3.5-Turbo-Instruct** language model.
- Use two datasets:
  - Instruction and Induction
  - GSM8K

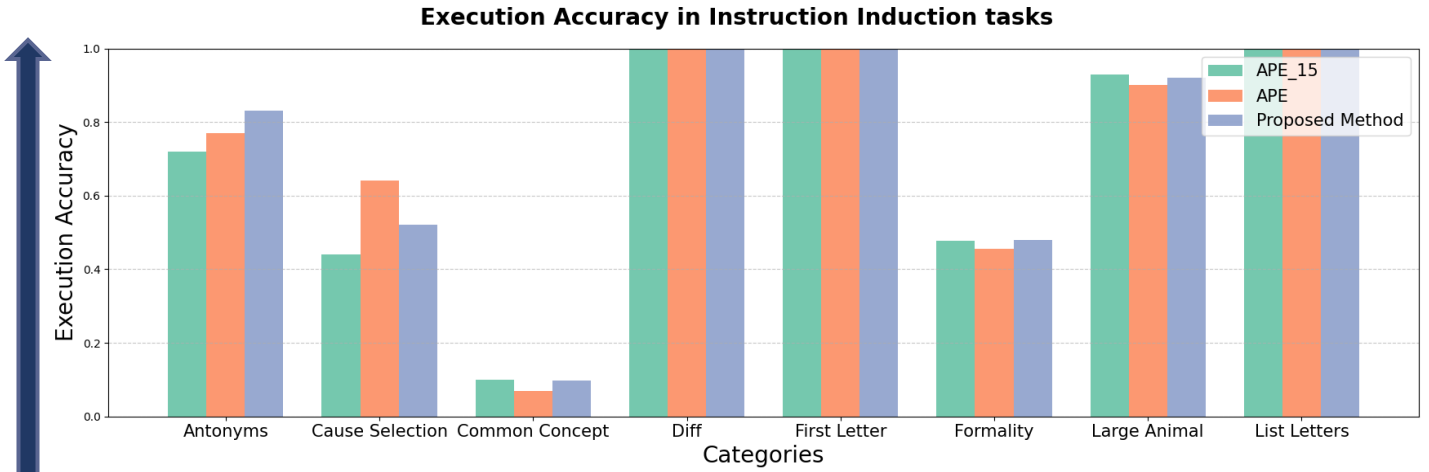
Experiment – Instruction and Induction

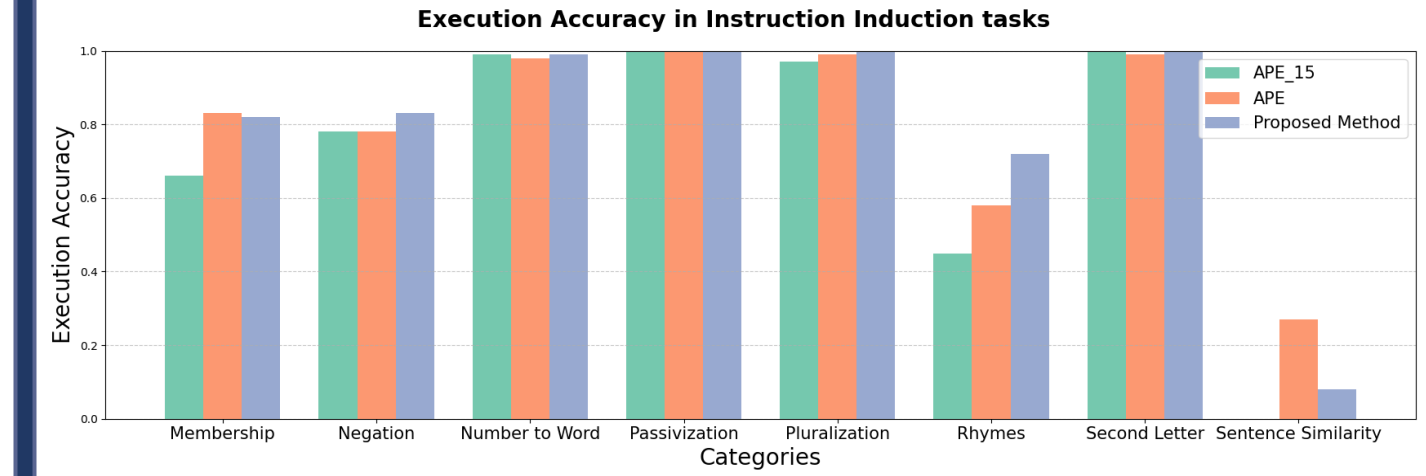
- **APE:**
  - Original parameter: k=90, 5 input-output pairs.
  - k=15, 3 input-output pairs
  - (k means the number of prompts generated initially)

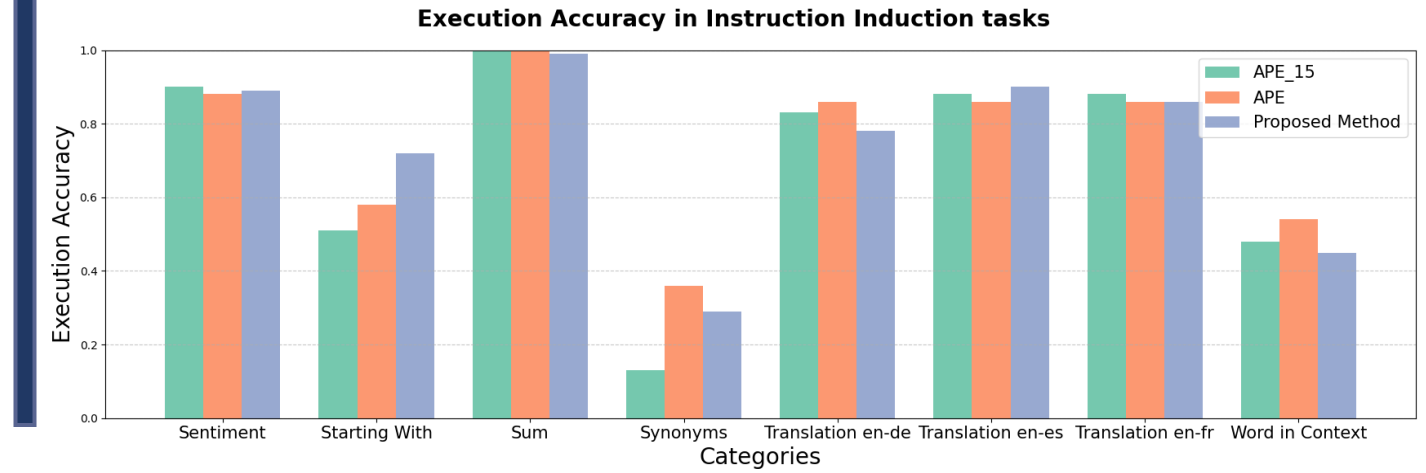
- **Proposed Method:**
  - k=15, 3 input-output pairs
  - **Regeneration Techniques:** APE-inspired Regeneration

- Allow a fair comparison across different prompt pool sizes.

Experiment-Instruction and Induction







APE_15: APE with k=15

APE: APE with k=90

Proposed Method: k=15

Tasks Where Our Method Performs Better:

- Lexical & Semantic Tasks
- Structural Transformations
- Word-Level Manipulation
- Complex Instruction Following

Tasks Where APE Performs Better:

- Arithmetic Tasks (Summation)

Experiment- Instruction and Induction

Why APE performs better than proposed method in Arithmetic Tasks (Summation)?

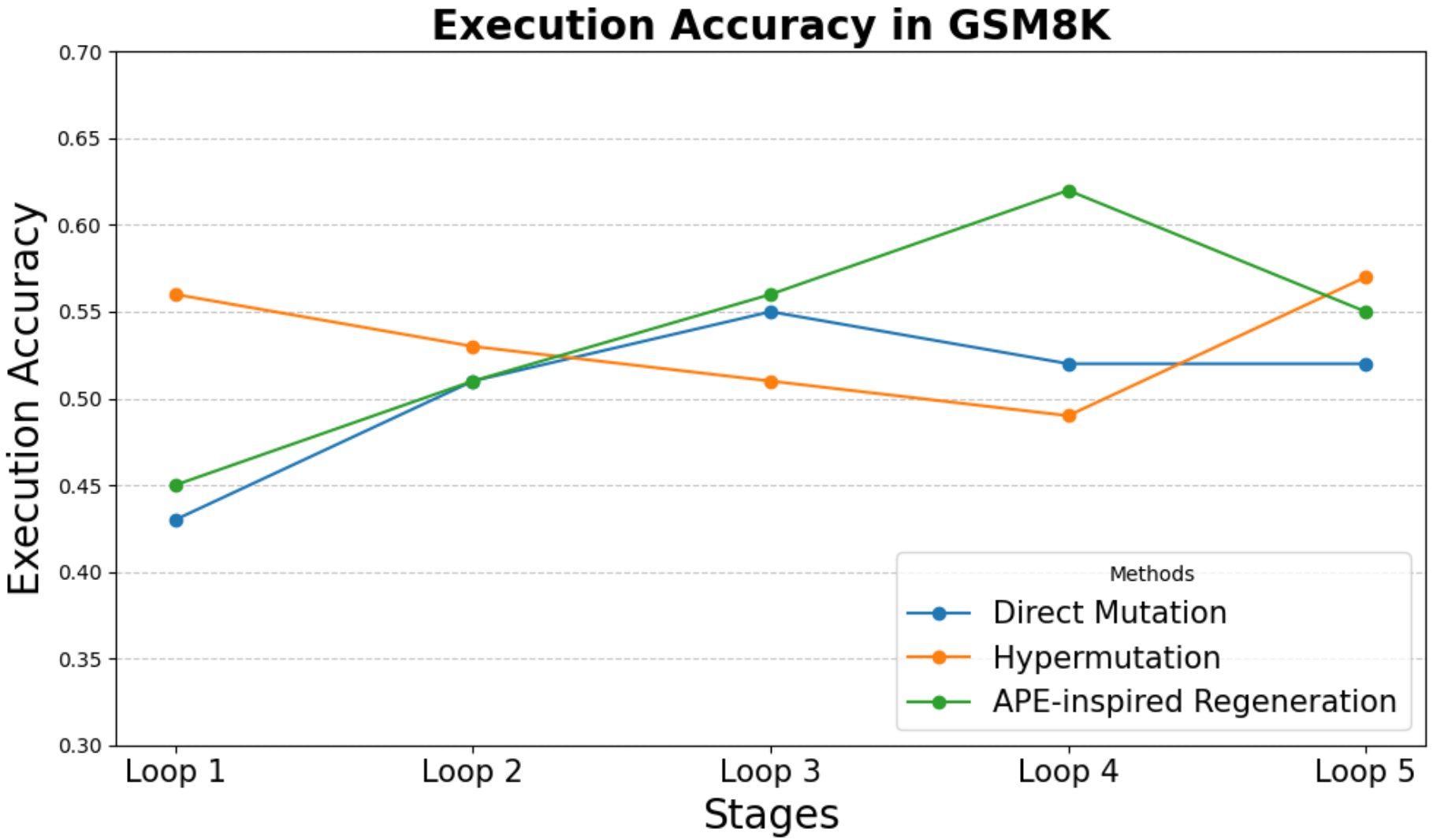
- Summation tasks benefit from **explicit, structured step-by-step reasoning**.
- **Why?** APE's **static high-scoring prompt selection** provides consistency without unnecessary variation.
- **Our Limitation:** Regeneration introduces **prompt variation**, which may not always be needed for arithmetic calculations.

Experiment- GSM8K

- The impact of the **number of iterations** of the **regeneration** strategy on the execution accuracy in the GSM8K dataset.(Step 3.1)

- **Proposed Method:**
  - All methods generate an initial set of k=15 prompts, 3 input-output pairs
  - **Loop 1-5:** Our method undergoes iterative regeneration, refining prompts progressively across five loops.
  - **Regeneration Techniques:**
    Direct Mutation, Hypermutation, APE-inspired Regeneration

Experiment- GSM8K



Loop1-5:
Proposed method with different regeneration loops

- The accuracy first increase in the early loops.
- The accuracy starts to decline in later loops

Experiment- GSM8K

The reason of the decline in accuracy may be:

- **Prompt Over-Optimization:**

  Prompts may become overly specialized to the input-output pairs used during regeneration

- **Reduced Diversity:**

  As loops progress, prompt diversity may diminish due to repeated refinement of high-scoring prompts, limiting adaptability.

Contents

Conclusion

- **Our Method:** A structured prompt optimization approach that integrates **multi-stage selection and regeneration**, balancing **quality improvement and diversity exploration** through **three regeneration Strategies.**

- **Key Benefits:**
  - **Balanced Optimization:** Enhances prompt quality while maintaining diversity.
  - **Improved Adaptability:** Excels in semantic reasoning, structural transformation, and instruction following tasks.
  - **Scalability & Efficiency:** Reduces computational overhead while maintaining high accuracy.

Conclusion

- **Future Directions:**
  - **Adaptive Iteration Control** to optimize regeneration loops.
  - **Enhanced Scoring Mechanisms** for better prompt ranking.
  - **Hybrid Regeneration Strategies** to further refine prompt evolution.
  - **Expansion to More Task Domains**, including multi-modal and reasoning-intensive tasks.

References

[1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022.

[2] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023.

[3] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim RocktÅNaschel. Promptbreeder: Self-referential self-improvement via prompt evolution. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024.

References

[4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. CoRR, abs/2110.14168, 2021.

[5] Or Honovich, Uri Shaham, Samuel R. Bowman, and Omer Levy. Instruction induction: From few examples to natural language task descriptions. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023, pages 1935–1952. Association for Computational Linguistics, 2023.